

Compressing Neural Networks via Regularization

Vilmos Vörös, HW921K

May 31, 2026

Contents

1	Background	2
1.1	Global Magnitude-based Pruning	3
1.2	Weight and Learning Rate Rewinding	3
1.3	Sparsity-enhancing Regularization	3
1.4	Iteratively Reweighted l_1 Minimization	4
2	Related Work	6
3	Methodology	6
3.1	IRL1 Regularization	6
3.2	Scheduling ϵ and λ	9
3.3	Robustness Metrics	10
4	Experiments	10
4.1	Length of Pruning Process	11
4.2	Sparsity-Accuracy Tradeoff	14
4.3	Sparsity-Robustness Tradeoff	14
5	Summary	15
A	Appendix	17

Introduction

Modern deep learning architectures (e.g BERT, Vision Transformers) tend to have billions of parameters and are trained on expensive and resource-intensive GPU clusters. Thus, the capacity need of storing these models and the latency their inference induces become bottlenecks of deploying them in resource-constrained environments and latency-sensitive applications. Besides designing more efficient architectures and training them from scratch, there are several well-established techniques of compressing a given model after training, such as network pruning, quantization, knowledge distillation and tensor decomposition.

My work focuses on network pruning which involves removing weights or groups of weights from the network whose contribution to the accuracy of the model is considered the smallest. I examined the effect of using three different regularization terms (L0, L1, weighted L1) in the objective function, and compared their capability of training a sparse network which retains accuracy when pruned to various degrees.

1 Background

Large neural networks have produced state-of-the-art results in numerous fields such as computer vision or natural language processing. To deploy these neural network model in an industry application, the most common option is training the model in a data center utilizing huge GPU clusters, liquid cooling, etc. At inference time, the clients could also access the model in the data center via internet.

However, with the proliferation of Internet-of-Things (IoT) devices and applications, neural network models are required to run more efficiently on resource-constrained devices to facilitate the growing number of edge AI use cases [1]. Efficiency or compactness refers to the model giving outputs accurate enough to enable the specific application, while also inducing low memory and energy-consumption overhead. The latter two translates to the model having a small parameter count and an architecture which minimizes latency and floating-point operations (FLOPs). Depending on the deployment scenario, a model may receive inputs that are far from the distribution of the training data or inputs that are manipulated to make the model produce a wrong output. Robustness of the model refers to its ability to maintain its performance even on such inputs. To sum up, a trained neural network has to satisfy accuracy, compactness and robustness requirements in order to become qualified for industrial applications.

In academia, there has been extensive research to improve them, but most works tackled them separately [2] [3]. To improve accuracy, the general approach was training overparameterized networks with regularization. Model compression techniques (i.e. weight pruning, knowledge distillation) and novel architecture have been proposed to achieve compactness while retaining accuracy. Finally, increasing robustness has been achieved by augmenting the training dataset to better represent the distribution of inputs at inference [4].

However, meeting these requirements simultaneously - so not increasing one at the expense of another - was found to be a challenge [5]. Therefore, it is important to evaluate a technique tackling one of the three requirements on all three aspects. In my case, measuring the impact of model compression on accuracy and robustness as well. In the following

sections, we give an overview of the techniques relevant to this work.

1.1 Global Magnitude-based Pruning

There are many contemporary methods for model compression. The main four categories include network pruning, knowledge distillation, tensor decomposition, and quantization [6], although it is also common to propose hybrid strategies. Network pruning refers to evaluating weights or groups of weights of the model based on a certain importance criteria and removing the ones which were considered the least important. This importance signal could be weight magnitude, the sensitivity of loss with respect to the weight, or other heuristics. After pruning, a fine-tuning or retraining phase is applied using the remaining weights in order to refit the pruned model to the training data. When the magnitude is utilized for representing the importance of weights, the pruning step uses a threshold for comparison, and every weight with magnitude below the threshold gets removed. Global pruning refers to having only one threshold and using it across all layers, regardless of their shape and function (i.e. classifier layer, convolutional layer). If every layer has its separate, local threshold, the pruning is applied uniformly across the entire network, which can lead to sub-optimal sparsity and the increased complexity of tuning these thresholds for a deep network [3]. In contrast, global pruning enables layers with different roles (i.e. feature extractor, classifier) to contribute to the overall sparsity non-uniformly.

1.2 Weight and Learning Rate Rewinding

1.3 Sparsity-enhancing Regularization

In the field of deep learning, regularization aims at reducing the generalization error of the model, often at the expense of the training error [7]. This mostly involves some modification made to the training algorithm, the training dataset or the way the output model is used. One common regularization technique is adding a norm penalty term to the objective function which expresses a prior belief about the weights of the final network. Injecting some prior knowledge into the training process could also be employed via adding constraints to the optimization algorithm, augmenting the dataset by applying transformations on the existing samples which the model must be invariant to, or sharing weights across different parts of the network.

As the use of norm penalties for model compression purposes is an established practice, I used the same approach. The general form of the cost function C in the case of supervised learning can be written as:

$$C(W) = \sum_i \mathcal{L}(f(x_i, W), d_i) + \lambda \cdot \mathcal{H}(W) \quad (1)$$

where W denotes the parameters of the neural network, $\mathcal{L}$ denotes the loss function used for training, function f is the output of the network, x_i is the training sample and d_i is the corresponding target. The regularization term consists of a function of the weights denoted by $\mathcal{H}(W)$, and a coefficient λ which determines the strength or importance of regularization compared to the performance of the model on the training samples.

In case of network pruning, our general assumption is that there is a sparse subnetwork inside the dense neural network which could achieve the same accuracy as the dense network when retrained in isolation (which means training it further after dropping the rest of the units from the dense network) [8]. Hence, the penalty term must represent the belief that the number of nonzero weights in the initial dense network should be as high as possible. Although finding the aforementioned sparse subnetwork is the primary goal of network pruning, the penalty term should not dominate the loss term in the objective to enable learning the task at hand.

1.4 Iteratively Reweighted l_1 Minimization

As L0 regularization results in a non-differentiable cost function, it is common to use convex relaxations of it, such as L1 regularization. Besides L1, there are non-convex functions which better approximate the L0 term, such as the log-sum function visualized in Figure 1.

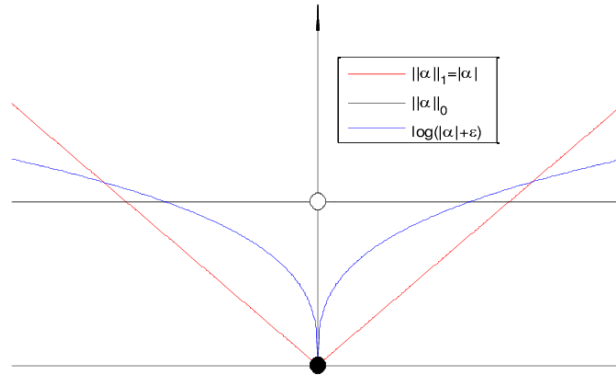


Figure 1: Comparison of L0, L1 and log-sum functions.

As the figure shows, the *log-sum* function approximates the L0 penalty term better; it penalizes smaller weights more, as the magnitude of the gradient increases as it approaches zero (from both sides). Boyd et. al. [9] proposes an iterative algorithm for reweighted L1 minimization, which will be referred to as *Iterative Reweighted l_1 Minimization* (IRL1) in this document. The subject of this paper is Compressed Sensing which is a subfield of signal processing aiming to reconstruct a signal vector $x_0 \in R^n$ from measurements $y = \Phi x_0$ where $\Phi \in R^{m \times n}$ and $m < n$. As there are fewer measurements than the dimension of the unknown signal, there are infinite number of possible solutions to x_0 . The main result of compressed sensing is that the signal x_0 can be reconstructed if it is sparse in some domain. There are more conditions that have to be met to reconstruct the sparse signal, but these go beyond the scope of this work. The reconstruction is done by minimizing the L1 norm of x_0 while using equations of $y = \Phi x_0$ as constraints. Authors aimed to improve upon the utilization of the L1 norm in terms of the sparsity achieved in the reconstructed signal. They showed that solving a sequence of weighted L1 minimization problems finds a local minimum of the concave log-sum function. As previously mentioned, the log-sum function is a better approximation of the L0 term, so it is more effective in sparsifying the reconstruction

than the L1 term. Thus, the adoption of this algorithm on the regularization term used for network pruning could potentially find a smaller subnetwork in the initial dense network.

As the experiments will be conducted on a convolutional network, the use of IRL1 is described in terms of weight tensors of a convolutional layer. The weighted L1 term can be written as:

$$\mathcal{H}(W^{(\ell)}) = \sum_{l,i,j} \sum_{u,v} A_{i,j,u,v}^{(\ell)} \odot |W_{i,j}^{(\ell)}(u,v)| \quad (2)$$

where $A^{(\ell)}$ is the tensor of weights used to balance the penalty induced by network weight magnitudes in layer ℓ . It has the same dimension as the weight tensor of the convolutional layer and $\odot$ denotes the element-wise multiplication between tensors $A^{(\ell)}$ and $W^{(\ell)}$. As both the weights of the network and the weights of the regularization get updated, let k denote the current iteration. Before the first iteration, all regularization weights are initialized as 1. Then after a few gradient descent steps, these weights are updated as follows:

$$A_{i,j,u,v}^{(\ell,k+1)} = \frac{1}{|W_{i,j,u,v}^{(\ell,k)}| + \epsilon} \quad (3)$$

where ϵ represents a small nonzero number which prevents division by zero and balances the extent to which small weights are penalized.

Figure 2 depicts three final weight distributions of a convolutional layer corresponding to IRL1 runs with different ϵ values. The additional distribution at the left shows the final distribution when training the layer without any regularization.

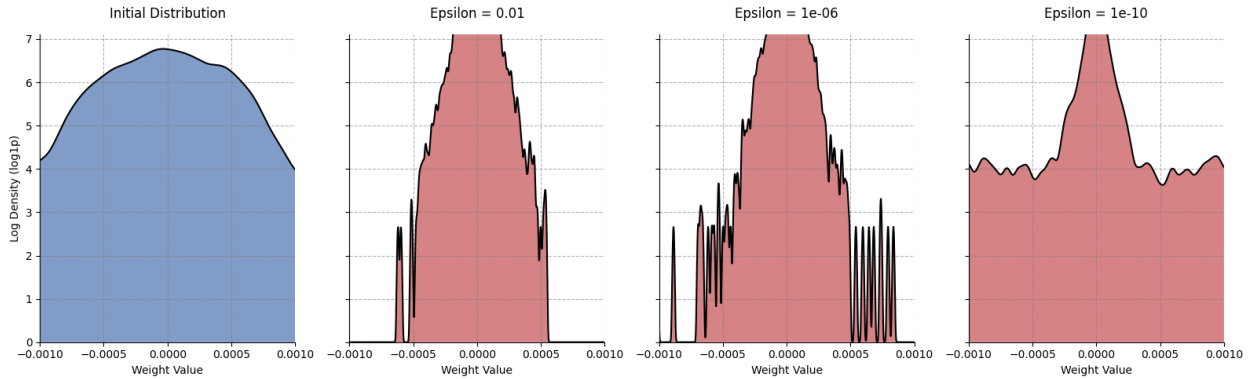


Figure 2: Final weight distributions of a convolutional layer corresponding to IRL1 runs with different ϵ values.

Not surprisingly, the distribution in blue resembles a normal distribution. IRL1 regularization with the largest value of ϵ pushed the whole distribution towards zero, similarly to L1 regularization. On the other hand, the rightmost distribution corresponding to the smallest ϵ value has produces a spike in a small zero-centered interval, thus resembling the application of a hard thresholding operator. The hard thresholding operator is usually how L0 regularization is implemented in the context of network pruning. Hence, the greater the value of ϵ , the better the IRL1 algorithm resembles L1 minimization, while smaller ϵ values make the effect of IRL1 more similar to L0 minimization.

The experiments done in [9] kept ϵ constant, but dynamically adjusting it to account for the dynamics of training could also be a valid strategy.

Another hyperparameter of this method is the frequency of updating tensor A . There are no guidelines for it in [9], as it is done after convergence of the weighted L1 minimization. In our case, the non-convex objective function of training the neural network never converges, so an update frequency has to be set heuristically. If the L1 term is reweighted too often, it may destabilize training as the optimizer would face a frequently-changing objective function. On the other hand, if the update interval is too long, the optimization may find a dense solution with training error too low for the reweighted L1 term to induce sparsity. Different sparsity curves - captured during the proposed pruning process in Chapter 3 - corresponding to different update frequencies are compared in Appendix A to provide some intuition on the effect of this hyperparameter.

2 Related Work

3 Methodology

In this section we present the proposed gradual pruning strategy utilizing the Iteratively Reweighted l_1 Minimization algorithm presented in Section 1.4. Additionally, we describe two design choices and their motivations in the pruning process.

3.1 IRL1 Regularization

As explained in previous sections, utilizing a sparsity-enhancing regularization term in the cost function forces the optimization process to find a sparser model than the initial one while fitting the data. The IRL1 algorithm enables us to approximate the most effective regularization term, the l_0 norm penalty term using gradient descent optimization. Regarding weight pruning methods, applying a global threshold for pruning facilitates sparser but accurate models due its flexibility in enhancing the sparsity of layers with different positions in the network unevenly. By applying the threshold multiple times, the initial dense model can be gradually compressed to a desired sparsity level.

Algorithm 1 blends these two techniques into a single iterative pruning framework. The algorithm consists of two phases. The first phase is a pre-training phase where the initial dense model is trained using the standard cross-entropy loss function for e epochs. After this, the resulting weights $\hat{W}$ - and optimizer specific buffers - are saved as a checkpoint

The second phase is the main iterative pruning phase, where the model is gradually compressed to the target sparsity level P . It consists of two sub-phases. The first one is defined as the regularization phase, where the model is trained using the cost function of Eq. 1 with the weighted l_1 regularization term defined in Eq. 2. The penalty weights $A^{(\ell)}$ are updated after n consecutive epochs, according to Eq. 3.

To eliminate the hyperparameter related to the length of the regularization phase, an early-stopping criterion is applied based on the difference in sparsity Δp_t between two consecutive epochs. If the increase in sparsity is smaller than a predefined tolerance ϵ for m consecutive epochs, then the regularization phase is stopped and the second sub-phase starts.

After updating the binary mask Z based on the global threshold τ and the regularized weights W , a re-fitting iteration is applied to let the remaining weights adapt to the new structure and recover accuracy. After the convergence of the latter training, the achieved sparsity is compared to the target sparsity P . If the current sparsity p is below the target, Phase 2 is repeated.

As discussed in Section 1.2, rewinding the remaining weights after pruning to states from a dense training checkpoint can lead to better performance than fine-tuning the pruned model from its current state. Therefore, the proposed algorithm restores the weights to checkpoint $\hat{W}$ after each pruning step, while also applying the newly discovered mask Z . Note that this rewinding step is one of the design choices whose benefits and drawback we evaluated during the experiments.

Algorithm 1 Iterative Pruning Framework

Input:	Training dataset $\mathcal{D}_{train}$, Validation dataset $\mathcal{D}_{val}$ Target sparsity P , Patience epochs x , Sparsity tolerance E Global magnitude threshold τ Update interval n ,
Output:	Pruned weights W^* , Final binary mask Z^*

Phase 1: Dense Pretraining

- 1: Initialize dense weights $W^{(0)}$
- 2: Train $W^{(0)}$ minimizing cross-entropy loss $\mathcal{L}_{CE}$ on $\mathcal{D}_{train}$
- 3: At epoch e , save current weights to checkpoint $\hat{W}$

Phase 2: Iterative Pruning

- 4: Initialize current sparsity $p \leftarrow 0$, binary mask $Z \leftarrow \mathbf{1}$, and penalty $A \leftarrow \mathbf{1}$
- 5: **while** $p < P$ **do**
Phase 2.1: Regularization
 - 6: Mini-batch training of W on $\mathcal{D}_{train}$ minimizing $\mathcal{L} = \mathcal{L}_{CE} + \sum(A \odot |W|)$
 - 7: Update penalty A according to Equation 3
 - 8:
 - 9: **if** $\Delta p_t = (p_t - p_{t-1}) < \epsilon$ for x consecutive epochs **then**
 - 10: **break** mini-batch training
 - 11: **end if***Phase 2.2: Mask Update & Optimization*
 - 12: $Z \leftarrow (|W| > \tau)$
 - 13: Restore and mask checkpoint weights: $W \leftarrow \hat{W} \odot Z$
 - 14: Optimize weights: $W \leftarrow \arg \min_W \mathcal{L}_{CE}(W, \mathcal{D}_{train})$
 - 15: $p \leftarrow 1 - \frac{\|Z\|_0}{N}$ (where N is the total number of parameters)
 - 16: **end while**
 - 17: **return** W, Z

Table 1: Algorithm: Iterative Pruning with Weighted L_1 Regularization

3.2 Scheduling ϵ and λ

Besides weight rewinding, the other design choices we focused on were the iterative decaying of ϵ and iterative increasing of λ parameters. The motivation behind this scheduling is to achieve a smoother transition from training on just the fidelity term to training on the cost function with the additional regularization term. By gradually increasing the pressure that regularization imposes on the weights, we can avoid the optimization process being overwhelmed by the regularization term at the beginning of the regularization phase.

To visualize the effect of introducing this smooth transition, we inspected the dynamics of the thresholded sparsity curve during the gradual pruning process. The thresholded sparsity is defined as the percentage of weights that are below the global threshold τ at each epoch.

Figure 3 shows the different dynamics in the thresholded sparsity of the whole model among the aforementioned two regularization strategies.

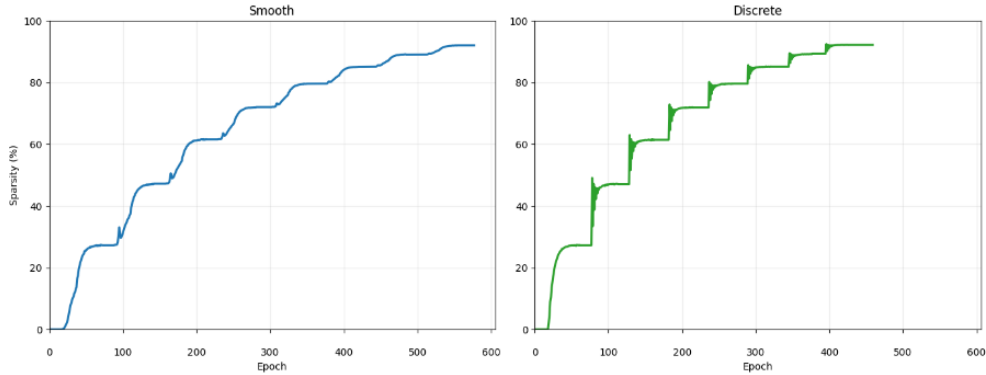


Figure 3: Scheduling of ϵ and λ during the training process.

We can conclude that continuously decaying ϵ and increasing λ leads to a more stable optimization process. Based on the dynamics of the sparsity curve, scheduling these parameters achieves a smoother increase, while the discrete step makes the sparsity oscillate. The latter is due to gradients from the fidelity term being close to zero at the beginning of the regularization phase, as the weights are converged to a local minimum by the end of the re-fitting phase. Hence, in the first mini-batch update of the first epoch of regularization the optimizer has a history of near-zero gradients so the optimizer takes a large, unnormalized step in the direction of the regularization term gradient, while neglecting the fidelity term gradient and pushing a large group of weights below the threshold. Then, in the next mini-batch update, the data loss gradients are no longer close to zero, and they pull the weights back up. By the end of the first regularization epoch, this oscillation is dampened by the optimizer’s momentum, but it still leads to a significant jump in the thresholded sparsity. As the regularization phase continues, weights near the threshold are pushed below and above the threshold alternately, leading to the observed oscillation in the sparsity curve.

On the other hand, the scheduling strategy eliminates the effect of unnormalized gradient steps and achieves a slower but more stable increase in sparsity.

If our assumption is that a smoother increase in sparsity indicates that regularization does not overwhelm the fidelity term of the loss function, and enables the optimization

process to find a better sparser architecture, then the scheduling strategy should achieve higher accuracy at the same sparsity level than its discrete counterpart.

3.3 Robustness Metrics

To facilitate a broader comparison of the two strategies, we utilized two complementary metrics regarding model robustness. The first one measures the smoothness of the input-output mapping the network implements.

Let $f(x; W) \in \mathbb{R}^C$ denote the neural network’s output logits for a given input sample $x \in \mathbb{R}^D$, parameterized by the weights W . To evaluate the smoothness of the learned input-output mapping, we analyze the sensitivity of the network to local input perturbations.

We define the expected smoothness metric, $\bar{\mathcal{J}}_{\text{smooth}}$, as the average Frobenius norm of the input Jacobian across the dataset $\mathcal{D}_{\text{train}}$:

$$\bar{\mathcal{J}}_{\text{smooth}} = \mathbb{E}_{x \sim \mathcal{D}_{\text{train}}} \left[\left\| \frac{\partial f(x; W)}{\partial x} \right\|_F \right] \quad (4)$$

A lower value of $\bar{\mathcal{J}}_{\text{smooth}}$ implies a lower upper bound on the empirical Lipschitz constant, indicating that the network implements a smoother, more robust mapping from the input space to the output logits.

The second metric aims to quantify how the learned weight matrices of the network amplify noise or small changes in the input.

Let W be the weight matrix of a given layer. The effective condition number, denoted as $\kappa_{\text{eff}}(W)$, is defined as the ratio of the largest singular value to the smallest non-zero singular value of W :

$$\kappa_{\text{eff}}(W) = \frac{\sigma_{\max}(W)}{\sigma_{\min}^+(W)} \quad (5)$$

where $\sigma_{\max}(W)$ is the maximum singular value, and $\sigma_{\min}^+(W)$ represents the smallest strictly positive singular value, ensuring numerical stability when evaluating sparse matrices where $W = \hat{W} \odot Z$.

Feng et al. [10] examined the effect of sparsity on the condition number and the Lipschitz constant of the network to capture the resulting robustness of the sparse model. They note that highly pruned models tends to have a lower upper bound of the Lipschitz constant. However, sparse weight matrices are more likely to be ill-conditioned, which is signaled by a high condition number.

By tracking these two metrics during the gradual pruning process, we can compare the effects of the aforementioned design decisions on the robustness of the resulting sparse model, besides the standard accuracy metric and the length of the process.

4 Experiments

In the following section we present the experimental results of the evaluation of the proposed pruning algorithm and its design choices. We first describe the setup of the experiments, including the architecture, the dataset, and the hyperparameters of the training.

Our benchmark task was image classification on the CIFAR-10 dataset [11], which consists of 60,000 32x32 color images in 10 classes, with 6,000 images per class. The dataset is split into a training set of 50,000 images and a test set of 10,000 images.

The utilized classification model was ResNet20 [12] which is presented in Table 2. $C(n)$ denotes 2D convolutional layers with n output channels, BN denotes batch normalization layers, GAP represents a Global Average Pooling layer, while $FC(n)$ denotes a fully connected layer with n output units. To mitigate the vanishing gradient problem, the architecture includes residual connections in each residual block, which are denoted by square brackets.

$C(16)\text{-BN}, [C(16)\text{-BN}, C(16)\text{-BN}] \times 3, [C(32)\text{-BN}, C(32)\text{-BN}] \times 3,$ $[C(64)\text{-BN}, C(64)\text{-BN}] \times 3, GAP, FC(10)$
--

Table 2: Architecture of ResNet20

For gradual pruning, we applied a custom strategy presented in Section 1.4. The penalty update (Eq. 3) was applied after every epoch, while the early stopping criterion in the regularization phase was applied with a tolerance E of 0.5% and a patience of 25 epochs. The global pruning threshold τ was set $1e - 5$ which resulted in an average of 20% increase in sparsity after each pruning step.

We examined four variations of the pruning process based on the design choices described in Chapter 3. The iterative decaying of ϵ and increasing of λ - which is referred to as the *smooth* method - was following a sigmoid function with steepness of 10 and a center of 0.6. On the other hand, the *discrete* method was defined by setting ϵ and λ to their final values at the first epoch of the regularization phase. The initial and final values of these parameters are listed in Table 4 in Appendix A.

Regarding the weight rewinding design choice, the *rewind* strategy is described as Step 13 in Algorithm 1, while the *no-rewind* strategy is defined by skipping this step and letting the remaining weights to be updated from their current state during the refitting phase.

Note that regularization term was defined on individual weights, not groups of weights. The additional hyperparameters are listed in Table 4.

4.1 Length of Pruning Process

To compare feasibility these four variations, we executed 15 runs of each variation with a different random seed and calculated metrics related to the length of the pruning process. The sample mean and standard deviation of each metric are listed in Table ???. Each training was run until a target sparsity level of 90% was reached. *Total epochs* refers to the entire length of the process, while refitting and regularization lengths refer to the average number of epochs spent in Step 6 and Step 14 of Algorithm 1, respectively. *Pruning steps* denote the average number of updating the binary mask Z in Step 12. At the end of the process, the final model with sparsity just above the target sparsity is evaluated on the test set. This final actual sparsity and top-1 accuracy is reported in the last two columns.

By comparing the different length metrics across the smooth and discrete methods, we can see that the scheduling of ϵ and λ leads to significantly longer overall length. This is

primarily because of the longer regularization phases, which indicates that the convergence of the thresholded sparsity takes longer. This is also visualized in the training curves of Figure 3. Despite the longer regularization phase, the smooth methods does not achieve significantly higher test accuracies. More specifically, in the case of the no-rewind strategy, discrete method seems to be more effective in maintaining accuracy, while the smooth methods performs better in the case of the rewind strategy.

Transition	Rewinding	Total Epochs	Pruning Steps	Refitting Length	Regularization Length	Sparsity (%)	Test Accuracy (%)
Smooth	Rewind	959.13 $\pm$ 217.68	12.67 $\pm$ 3.42	18.42 $\pm$ 0.46	56.93 $\pm$ 3.62	91.48 $\pm$ 0.46	67.25 $\pm$ 1.18
	No Rewind	1251.53 $\pm$ 271.06	12.67 $\pm$ 3.42	42.21 $\pm$ 4.07	56.93 $\pm$ 3.68	91.48 $\pm$ 0.46	69.47 $\pm$ 3.22
Discrete	Rewind	638.13 $\pm$ 138.17	12.67 $\pm$ 3.42	18.58 $\pm$ 0.65	31.20 $\pm$ 2.61	91.54 $\pm$ 0.49	66.62 $\pm$ 0.90
	No Rewind	925.60 $\pm$ 187.26	12.67 $\pm$ 3.42	42.41 $\pm$ 3.96	30.85 $\pm$ 2.50	91.55 $\pm$ 0.44	71.06 $\pm$ 2.12

Table 3: Performance and Efficiency Metrics across Pruning Configurations (Mean $\pm$ Standard Deviation). Results are aggregated over 15 runs.

Regarding the difference caused by weight rewinding, we can observe that weight rewinding reduces the length of the refitting phase by more than 50%. A bit contradicting to the common belief that weight rewinding leads to better performance, the no-rewind strategy achieves higher test accuracies in both cases of applying smooth or discrete transition between the refitting and regularization phases. On the other hand, we may conclude that the benefit of reducing the length of the refitting phase by weight rewinding outweighs the ca. 2 – 5% degradation in test accuracy.

4.2 Sparsity-Accuracy Tradeoff

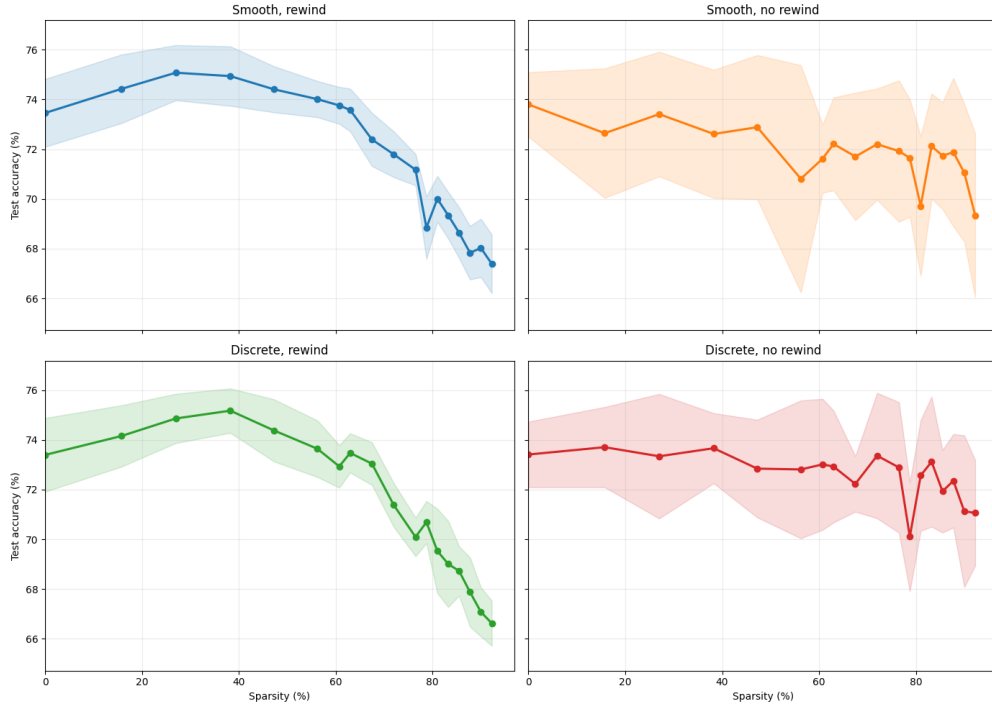


Figure 4: Test accuracy of pruned versions of ResNet20 on the CIFAR-10 dataset, pruned by four variations of the proposed pruning strategy.

4.3 Sparsity-Robustness Tradeoff

Besides the test accuracy across the pruning stages, we compared the different variations of the pruning process along metrics related to model robustness. The definition of the average Jacobian Norm of a given model is presented in Equation ?? from the previous Chapter. To capture how pruning changes the smoothness of the model’s input-output mapping, we evaluated the same 15 runs of each pruning variation during their training process and computed this metric. Figure 5 shows how the average Jacobian norm changed across the sparsity levels. Note that models were evaluated after the refitting phase.

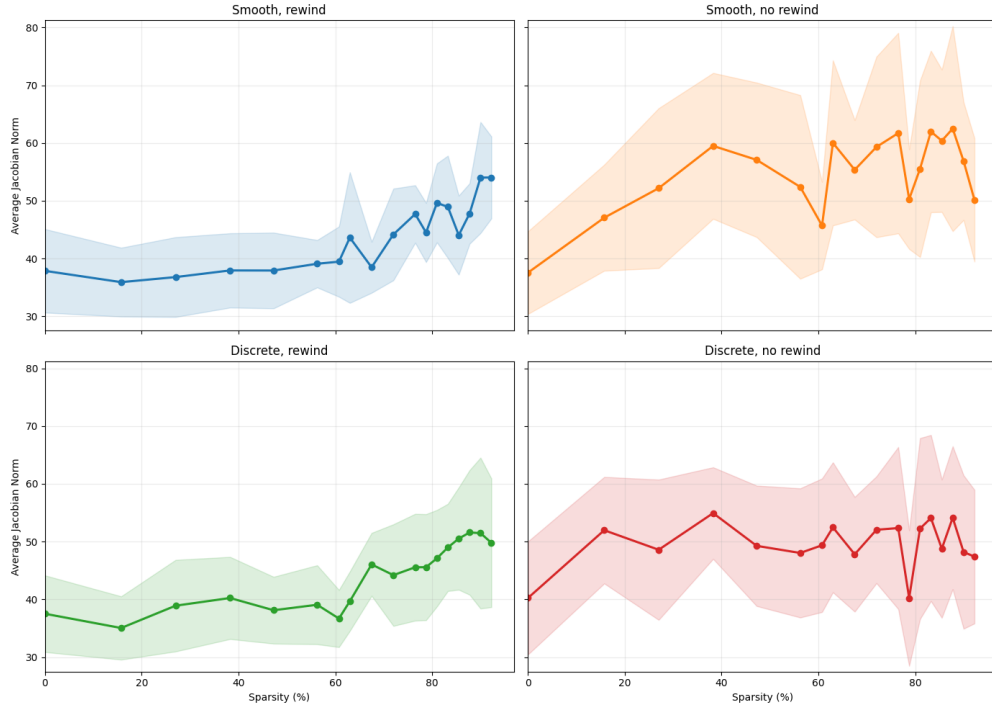


Figure 5: Average Jacobian norm of pruned versions of ResNet20 on the CIFAR-10 dataset.

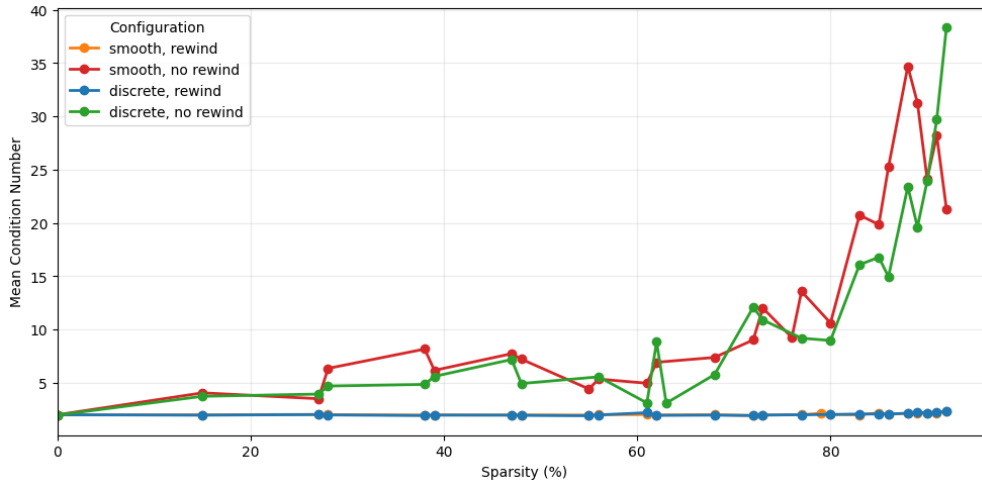


Figure 6: OOD accuracy of pruned versions of ResNet20 on the CIFAR-10-C dataset.

5 Summary

To sum up, the presented methods for measuring the impact of pruning on OOD and adversarial robustness of a neural network showed that the proposed gradual magnitude-based pruning strategy does not reshape the model’s robustness features. Evaluation on the CIFAR-10-C dataset indicated that the pruned models did not get more vulnerable

to common corruptions than the original dense model. Furthermore, when examining the progression of the Fourier sensitivity heatmaps during the pruning process, no significant change could be detected in the model’s robustness to perturbations of different frequencies and orientations, except at extreme sparsity levels. However, the distance to the decision boundary - approximated by the norm of the minimal adversarial perturbation found by the FAB attack - gradually decreased as sparsity levels increased. Based on these results, we hypothesize that our pruning process only decreases robustness to adversarial inputs, while maintains the OOD robustness of the model.

While the proposed pruning method did not worsen the OOD robustness, research suggests that model compression techniques like pruning can be enhanced to contribute to robustness targets as well during training. Therefore, future work is needed to find modifications of this specific pruning method to improve upon robustness, accuracy and compactness simultaneously.

References

- [1] R. Singh and S. S. Gill, “Edge ai: A survey,” *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023.
- [2] M. Tan and Q. V. Le, “Efficientnet: Rethinking model scaling for convolutional neural networks,” *CoRR*, vol. abs/1905.11946, 2019.
- [3] H. Cheng, M. Zhang, and J. Q. Shi, “A survey on deep neural network pruning: Taxonomy, comparison, analysis, and recommendations,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, no. 12, pp. 10558–10578, 2024.
- [4] K. Kireev, M. Andriushchenko, and N. Flammarion, “On the effectiveness of adversarial training against common corruptions,” *CoRR*, vol. abs/2103.02325, 2021.
- [5] J. Diffenderfer, B. R. Bartoldson, S. Chaganti, J. Zhang, and B. Kailkhura, “A winning hand: Compressing deep networks can improve out-of-distribution robustness,” *CoRR*, vol. abs/2106.09129, 2021.
- [6] J. O. Neill, “An overview of neural network compression,” 2020.
- [7] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. <http://www.deeplearningbook.org>.
- [8] J. Frankle and M. Carbin, “The lottery ticket hypothesis: Training pruned neural networks,” *CoRR*, vol. abs/1803.03635, 2018.
- [9] E. J. Candes, M. B. Wakin, and S. P. Boyd, “Enhancing sparsity by reweighted l1 minimization,” 2007.
- [10] Y. Feng, S.-H. J. Lin, B. Gao, and X. Wei, “Lipschitz constant meets condition number: Learning robust and compact deep neural networks,” 2025.

- [11] A. Krizhevsky, “Learning multiple layers of features from tiny images,” technical report, University of Toronto, 2009.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” *CoRR*, vol. abs/1512.03385, 2015.

A Appendix

Table 4: Hyperparameter Configuration for Training

Parameter	Value
Optimizer	Adam
Learning Rate	1×10^{-3}
Batch Size	64
Early Stopping Patience	8
Loss Function	Cross-Entropy
Activation	ReLU
Weight initialization	Uniform Xavier